

Unit 2: Python Control Flow, Functions & Modules • Topic 4

Functions — def, Parameters, Return Values, Scope, Lambda Functions

Writing reusable code blocks with functions: parameters, return values, variable scope, and lambda functions.

As programs grow, repeating the same block of code in many places becomes hard to manage. A **function** is a named, reusable block of code that performs a specific task. Functions make programs shorter, more organised, easier to test, and easier to read — an application of the **decomposition** idea from Unit 1 directly to code.

4.1 Defining and Calling a Function — def

A function is created using the `def` keyword, followed by the function name, parentheses (optionally containing parameters), and a colon. The indented block below it is the **function body**. Defining a function does not run it — it must be **called** by name to execute.

```
def greet():                # function definition
    print("Hello, welcome!")

greet()                    # function call - this is when it actually runs
greet()                    # can be called as many times as needed
```

4.2 Parameters and Arguments

A function can accept input values through **parameters** (the names listed in the function definition) which receive **arguments** (the actual values passed in when the function is called).

```
def greet(name):            # 'name' is a parameter
    print("Hello,", name)

greet("Kabir")              # "Kabir" is the argument passed in
greet("Meera")
```

Default Parameters

A parameter can be given a **default value**, used automatically when the caller does not supply that argument.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Kabir")              # Hello, Kabir! (uses the default)
greet("Meera", "Good morning") # Good morning, Meera!
```

Positional vs Keyword Arguments

```
def show_info(name, age):
    print(f"{name} is {age} years old")

show_info("Riya", 19)          # positional - order matters
show_info(age=19, name="Riya") # keyword - order does not matter
```

*args and **kwargs (brief introduction)

When the number of arguments is not fixed, *args collects any number of extra positional arguments into a tuple, and **kwargs collects extra keyword arguments into a dictionary.

```
def total(*args):
    return sum(args)

print(total(2, 4, 6))      # 12
print(total(1, 2, 3, 4, 5)) # 15
```

4.3 The return Statement

return sends a value back to the code that called the function, and immediately ends the function's execution. A function without a return statement (or with a bare return) implicitly returns None.

```
def add(a, b):
    return a + b          # sends the sum back to the caller

result = add(5, 7)        # result now holds 12
print(result)             # 12
print(add(3, 4) + 10)     # the returned value can be used directly: 17
```

print() vs return: print() only displays a value on screen and gives back None — it cannot be used in further calculations. return actually hands the value back so it can be stored or reused. result = print(5+5) makes result equal to None, not 10.

4.4 Scope of Variables

Scope determines where in a program a variable can be accessed.

- **Local scope** – a variable created inside a function exists only within that function and is destroyed once the function finishes running.
- **Global scope** – a variable created outside all functions, at the top level of the program, can be read from anywhere, including inside functions.

```
x = 10                      # global variable

def show():
    y = 5                   # local variable - only exists inside show()
    print(x, y)             # can READ the global x here

show()                      # 10 5
print(y)                   # NameError: y is not defined outside the function
```

By default, assigning to a variable inside a function creates a **new local variable**, even if a global variable of the same name exists. To modify a global variable from inside a function, use the global keyword.

```

counter = 0

def increment():
    global counter          # without this, Python would create a new LOCAL counter
    counter += 1

increment()
increment()
print(counter)              # 2

```

4.5 Lambda Functions

A **lambda function** is a small, anonymous (unnamed) function defined in a single line using the lambda keyword, useful for short, throwaway functions that are used only once or passed as arguments to other functions.

```

square = lambda x: x * x      # equivalent to: def square(x): return x*x
print(square(6))              # 36

add = lambda a, b: a + b
print(add(3, 4))              # 7

```

Common Use: as a key for sorting

```

students = [("Ravi", 82), ("Anya", 91), ("Zoya", 76)]
students.sort(key=lambda s: s[1])          # sort by marks (2nd item of each tuple)
print(students)  # [('Zoya', 76), ('Ravi', 82), ('Anya', 91)]

```

Aspect	Regular function (def)	Lambda function
Name	Has a name	Anonymous (can be assigned a name, but usually inline)
Body	Can contain multiple statements	Restricted to a single expression
return	Uses explicit return	The expression's value is returned automatically
Typical use	Reusable, complex logic	Short, one-off functions, e.g. as sort/key/filter arguments

Practice Questions

A. Multiple Choice Questions

- Which keyword is used to define a function in Python?
(A) function (B) def (C) func (D) define
- What does a function return if it has no explicit return statement?
(A) 0 (B) "" (empty string) (C) None (D) It raises an error
- A variable created inside a function, accessible only within it, has:
(A) Global scope (B) Local scope (C) No scope (D) Universal scope
- Which keyword allows a function to modify a global variable?
(A) global (B) local (C) nonlocal (D) static
- A lambda function's body is restricted to:
(A) Any number of statements (B) A single expression (C) A loop only (D) A print statement only
- In `show_info(age=19, name='Riya')`, the arguments are passed as:
(A) Positional arguments (B) Keyword arguments (C) Default arguments (D) *args

B. True / False

- Calling `print()` inside a function returns the printed value to the caller.
- A parameter with a default value can be omitted when calling the function.
- Local variables inside a function are destroyed once the function finishes executing.
- Lambda functions can contain multiple statements and a return keyword.

C. Short Answer Questions

- Differentiate between a parameter and an argument.
- Explain the difference between `print()` and `return` with an example.
- What is the difference between local and global scope? How does the `global` keyword help?
- What is a lambda function, and how does it differ from a regular function defined with `def`?

D. Long Answer / Descriptive Questions

- Write a function `calculate_grade(marks)` that returns a letter grade (A/B/C/D/F) using `if-elif-else` inside it, and explain how the return value can be used by the caller.
- Explain variable scope in Python with a program that demonstrates a `NameError` from accessing a local variable outside its function, and a correct use of the `global` keyword to fix a related issue.

E. Programming Problems

- Write a function `is_prime(n)` that returns `True` or `False` depending on whether `n` is a prime number, and use it to print all prime numbers between 1 and 50.
- Write a function `calculate_bill(units, rate=8)` that calculates an electricity bill given the number of units and an optional rate per unit (default 8), and call it with and without the rate argument.
- Write a lambda function that takes a number and returns `True` if it is even, else `False`. Use it with the built-in `filter()` function to extract even numbers from the list `[1..20]`.

20. Write a function `sum_of_squares(*numbers)` that accepts any number of arguments using `*args` and returns the sum of their squares.

Answer Key (A & B): MCQ: 1-B, 2-C, 3-B, 4-A, 5-B, 6-B | T/F: 7-False, 8-True, 9-True, 10-False